

```

import streamlit as st #Streamlit sirve para crear la interfaz que vamos a ocupar en la web
import pandas as pd #Pandas nos sirve para leer la base de datos que creamos en la tabla de excel
import geopandas as gpd #Geopandas es la extensión de pandas que lee datos geoespaciales (lo que usamos para el mapa de México)
from shapely.geometry import Point #Nos ayuda a manipular objetos geométricos. En este caso, creamos un polígono para definir las fronteras
#Point son las coordenadas de un punto en dos dimensiones
import matplotlib.pyplot as plt #Matplotlib nos ayuda a mostrar las gráficas
import random #Genera números aleatorios

random.seed(42) #Esta semilla nos sirve para asegurarnos de que los números aleatorios generados no cambien al volver a correr el archivo

import networkx as nx #Network genera los grafos
from collections import deque #Deque es la estructura de datos que usamos para la BFS
import threading #Threading sirve para poder correr funciones simultaneamente
import time #Time nos sirve para poder tener un control de como se van infectando los estados
import numpy as np #Numpy sirve para hacer operaciones matemáticas con vectores
import math #Las operaciones correspondientes

#python -m streamlit run EXAMEN_FINAL_IA.py comando para correrlo desde la terminal

#Traemos el archivo para representar el mapa (que encontramos en github)
gdf = gpd.read_file("https://raw.githubusercontent.com/rb-one/geodata_mx/refs/heads/main/geodata_mx_minified.json")
#Extraemos los datos de la tabla de Excel
datos = pd.read_csv("IA TABLA EXAMEN FINAL.csv")

class Estado:
    _counter = 0 #cuenta la cantidad de instancias, es un atributo privado de la clase (lo comparten todas las instancias)
    corriendo_hilos = True
    _threads = [] #aquí pondremos todos los hilos que se vayan creando

    def __init__(self, nombre, poblacion, lluvias_intensas = 0, altas_temperaturas = 0, bajas_temperaturas = 0,
                 sistema_salud = 0, densidad = 0, ganaderia = 0, pobreza = 0, respiratorio = 0, ind_contagio = 0.0):

        self._id = Estado._counter #atributo privado para que no se pueda modificar
        Estado._counter += 1 #comienzan en 0 para poder asociarlos a posiciones en una lista

        self.nombre = nombre
        self.poblacion = poblacion

        self.__hilo_enfermar = None #Conforme el virus se vaya propagando, la población va disminuyendo

        self.tiempo_contagio = 0
        self.historial=[0] #guarda la población enferma conforme pasa el tiempo

        #Los siguiente factores puede contribuir a la transmisión de una enfermedad
        #En cada instancia los ponderamos de 0 a 10 indica que tanto afectan la propagación
        #10 - 0
        self.lluvias_intensas = lluvias_intensas #poco-mucho
        self.altas_temperaturas = altas_temperaturas #bajo-alto promedio del mes con mayor temperatura al año
        self.bajas_temperaturas = bajas_temperaturas #alto-bajo promedio del mes con menor temperatura al año
        self.sistema_salud = sistema_salud #bueno-deficiente (evalúa la calidad y disponibilidad para todos)
        self.densidad = densidad #poca-mucha / densidad poblacion, aglomeraciones
        self.ganaderia = ganaderia #poca_mucha (las zoonosis son enfermedades que se transmiten mediante animales
vertebrados)
        self.pobreza = pobreza #poca-mucha
        self.respiratorio = respiratorio #bajo-alto evalpu el impacto que hantenido algunas enfermedades respiratorias como
la influenza, neumonia, covid

        self.sanos = self.poblacion//100000 #personas sanas proporcional a la cantidad de personas en el estado (El mayor es 100)
        self.enfermos = 0 #Las personas que noe están sanas están enfermas
        self.ind_contagio = ind_contagio #la velocidad de propagación del virus en el estado

        self.camino = "" #Aquí guardamos los estados por los que tuvo que pasar el virus
#Nos deja acceder al atributo privado ID sin que el usuario lo pueda modificar
@property
def id(self):
    return self._id

#Contagia a la población del estado paulatinamente
#PATO es el virus/patógeno
def enfermar(self):
    while self.sanos > 0 and Estado.corriendo_hilos:
        self.enfermos += math.ceil((1 - self.ind_contagio) * self.sanos)
        self.sanos = math.floor(self.sanos * self.ind_contagio)
        print(f"{self.nombre}: sanos={self.sanos}, enfermos={self.enfermos}")
        self.historial.append(self.enfermos)
        time.sleep(1)

```

```
#La función pasa a un segundo plano mientras sigue corriendo el resto del programa. Es decir, los estados siguen siendo infectados sin
necesidad de estar interactuando
def iniciar_hilo(self, pato):
    self.__hilo_enfermar = threading.Thread(target=self.enfermar)#Creamos el hilo del estado
    Estado._threads.append(self.__hilo_enfermar)#Lo guardamos junto con los hilos de los demás estados
    self.__hilo_enfermar.start()#Hasta que da start, empieza el hilo

#Corta desde la raiz el hilo de los estados. La vamos a llamar cuando todos los estados están contagiados.
@classmethod
def detener_hilos(cls):
    cls.corriendo_hilos = False
    for hilo in cls._threads:
        hilo.join()

# -----
# Definición de la clase Virus
# -----

class Virus:
    def __init__(self, nombre = "Virus1", lluvias_intensas = 0, altas_temperaturas = 0, bajas_temperaturas = 0,
        densidad = 0, ganaderia= 0, pobreza = 0, respiratorio = 0, estado_inicial = 0, paso_semanas= 1):

        self.nombre = nombre
        #Estos atributos son los que facilitan que el virus se propague. Guardan las características que vuelven sensible al estado
        self.lluvias_intensas = 10-lluvias_intensas
        self.altas_temperaturas = 10-altas_temperaturas
        self.bajas_temperaturas = 10-bajas_temperaturas
        self.densidad = 10-densidad
        self.ganaderia = 10- ganaderia
        self.pobreza = 10-pobreza
        self.respiratorio = 10-respiratorio

        #paso semanas es la velocidad de contagio entre estados(cuntas estados se contagian en una semana)
        if(paso_semanas == 1):
            self.paso_semanas = 2
        elif(paso_semanas == 2):
            self.paso_semanas = 3
        else:
            self.paso_semanas= 4

#El estado en el que empieza la propagación
        self.estado_inicial = estado_inicial

# -----
# Datos de Estado
# -----

#Los datos de nuestra tabla están normalizados. Asi que multiplicamos por 10 para poder lidiar con números más grandes
nombre_estados = np.array(datos['NOMBRE'])
poblacion_estados = np.array(datos['POBLACION_TOTAL']).astype(str)
lluvias_intensas = (1-np.array(datos['LLUVIAS_INTENSAS']))*10
altas_temperaturas = (1-np.array(datos['ALTAS_TEMPERATURAS']))*10
bajas_temperaturas = np.array(datos['BAJAS_TEMPERATURAS'])*10
sistema_salud = np.array(datos['SISTEMA_SALUD'])*10
densidad = (1-np.array(datos['DENSIDAD']))*10
ganaderia = (1-np.array(datos['GANADERIA']))*10
pobreza = (1-np.array(datos['POBREZA']))*10
respiratorio = (1-np.array(datos['INDICE_RESPIRATORIO']))*10

#Vamos a instanciar los 32 estados y cargar los atributos que nos da la tabla
estados = []
for i in range(len(datos)):
    cleaned_poblacion = int(poblacion_estados[i].replace(',','').split('.')[0])#En la tabla, esto venia con comas. Al eliminar estas, pudimos
    tratarlo como número
    estado=Estado(nombre_estados[i], cleaned_poblacion, lluvias_intensas[i], altas_temperaturas[i], bajas_temperaturas[i], sistema_salud[i],
densidad[i], ganaderia[i], pobreza[i], respiratorio[i])
    estados.append(estado)

# -----
# Fronteras
# -----

#Creamos la gráfica con sus fronteras terrestres y agregamos 3 aéreas (Nuevo León, Jalisco y CDMX).
mexico_grafo = {
    0: [31, 13], #Aguscalientes, Zacatecas, Jalisco
    1: [25, 2],
    2: [1],
    3: [30, 22, 26],
    4: [7, 9, 31, 18],
    5: [13, 15],
    6: [26, 29, 19],
    7: [25, 24, 4, 9],
```

```

8: [13, 14, 18, 16],
9: [7, 24, 17, 31, 4],
10: [13, 31, 23, 21, 15],
11: [16, 15, 14, 19, 20],
12: [23, 29, 20, 28, 14, 21],
13: [31, 17, 5, 10, 15, 0, 8],
14: [12, 15, 16, 20, 28, 8, 21, 11],
15: [5, 13, 10, 21, 14, 11],
16: [8, 14, 20, 11],
17: [31, 13, 9],
18: [23, 4, 27, 8],
19: [6, 29, 20, 11],
20: [29, 19, 12, 28, 14, 11, 16],
21: [14, 23, 12, 15, 10],
22: [30, 3],
23: [10, 29, 27, 18, 31, 12, 21],
24: [7, 17, 9, 25],
25: [24, 7, 1],
26: [3, 29, 6],
27: [18, 23, 29],
28: [12, 20, 14],
29: [27, 23, 12, 20, 19, 6, 26],
30: [22, 3],
31: [23, 9, 4, 10,13,0,17]
}

# -----
# Inicializar listas y sesión
# -----
#Vamos a pedirle al usuario que ingrese los datos del virus y guardarlos en virus actual
if "virus_actual" not in st.session_state:
    st.session_state.virus_actual = None

# -----
# Menú lateral de la interfaz
# -----
opcion = st.sidebar.selectbox(
    "Selecciona una opción:",
    ["Inicio", "Crear virus", "Simular"]
)

#Explicamos como funciona la página y el servicio que ofrecemos
if opcion == "Inicio":
    st.title("Simulación de Epidemiología Mexicana")
    st.write("Bienvenido al simulador de propagación de virus. Aquí puedes diseñar un virus y modelar su propagación en México.")
    st.write("Para comenzar, selecciona 'Crear virus' en el menú lateral.")

    #Mostramos el mapa de México
    fig, ax = plt.subplots(figsize=(12, 10))
    gdf.plot(ax=ax, edgecolor='black', color='lightgray', linewidth=0.5)
    ax.axis("off") # Esta línea es de matplotlib, no Streamlit

    st.title("México")
    st.pyplot(fig)

    #Mostramos México como un grafo, así estaremos trabajando
    G = nx.Graph(mexico_grafo) #creamos el grafo con la librería networks
    etiquetas = {nodo: estados[nodo].nombre for nodo in G.nodes()} #un diccionario con los nombres de cada nodo

    plt.figure(figsize=(12, 10))

    #Dibujamos el grafo
    pos = nx.spring_layout(G) #genera una posición para cada nodo intentando Mantenerlso separados
    nx.draw(
        G,
        with_labels=True,
        labels=etiquetas,
        node_size=1000,
        font_size=8
    )
    # plt.title("Conexiones entre Estados Mexicanos")

    st.pyplot(plt.gcf())
# -----
# Crear virus
# -----
elif opcion == "Crear virus":
    #Le pedimos al usuario que nos de los datos del virus
    st.header("Nombra a tu virus")

    nombre_virus = st.text_input("")

    st.subheader("Características:")
    st.info("Selecciona qué atributos favorecen la propagación del patógeno:")

    #Slider es la barra que se desliza para que el usuario escoga que condiciones del estado propician su propagación

```

```

lluvias_intensas = st.slider("Lluvias intensas", 0, 9, 0)
altas_temperaturas = st.slider("Altas temperaturas", 0, 9, 0)
bajas_temperaturas = st.slider("Bajas temperaturas", 0, 9, 0)
densidad = st.slider("Densidad", 0, 9, 0)
ganaderia = st.slider("Ganaderia", 0, 9, 0)
pobreza = st.slider("Pobreza", 0, 9, 0)
respiratorio = st.slider("Via Respiratoria", 0, 9, 0)

paso_semanas = st.slider("Velocidad de transmisión entre estados", 1, 3, 1)

```

```

st.subheader("¿En qué estado quieres comenzar?")
#estado_inicial = st.selectbox("Estado inicial", estados.id, "", nombre_estados)

```

```

#Mostramos una caja con las opciones de todos los estados y de ahí obtenemos el ID
#En el diccionario guardamos el nombre de la cajita (número y estado) y nos regresa el ID
opciones_estados = {f"{estado.id}. {estado.nombre}": estado.id for estado in estados}
seleccion_estado = st.selectbox("Estado inicial", opciones_estados.keys())
estado_inicial = int(opciones_estados[seleccion_estado])

```

```

#Creamos el nuevo virus con los datos guardados

```

```

if st.button("Guardar"):

```

```

    nuevo_virus = Virus(nombre_virus, lluvias_intensas, altas_temperaturas,
                        bajas_temperaturas, densidad, ganaderia, pobreza,
                        respiratorio, estado_inicial)

```

```

    st.session_state.virus_actual = nuevo_virus
    st.success(f"Virus '{nombre_virus}' guardado con éxito.")

```

```

    resistencia = []
    for i in range(len(estados)):

```

```

        resis = nuevo_virus.lluvias_intensas*estados[i].lluvias_intensas + nuevo_virus.altas_temperaturas*estados[i].
altas_temperaturas + nuevo_virus.bajas_temperaturas*estados[i].bajas_temperaturas + estados[i].sistema_salud + nuevo_virus.densidad*estados[i].
densidad + nuevo_virus.ganaderia*estados[i].ganaderia + nuevo_virus.pobreza*estados[i].pobreza + nuevo_virus.respiratorio*estados[i].
respiratorio

        resistencia.append( int(resis))

```

```

#Normalizamos

```

```

denominador = max(resistencia)
min_norm = min(resistencia)

```

```

for i in range(len(estados)):

```

```

    n = (resistencia[i]-min_norm)/denominador
    if n>=.99:
        n = .99
    elif n<=.7:
        n = .7

```

```

    estados[i].ind_contagio = n

```

```

#Organizmos los vecinos de cada estado según su resistencia, para que se contagie primero el más débil

```

```

for k in mexico_grafo.keys():
    mexico_grafo[k].sort(key = lambda i: resistencia[i])

```

```

else:

```

```

if st.session_state.virus_actual is None:
    st.error("Por favor, crea un virus antes de iniciar la simulación.")
else:

```

```

    nuevo_virus = st.session_state.virus_actual
    resistencia = []

```

```

    for i in range(len(estados)):

```

```

        resis = nuevo_virus.lluvias_intensas*estados[i].lluvias_intensas + nuevo_virus.altas_temperaturas*estados[i].
altas_temperaturas + nuevo_virus.bajas_temperaturas*estados[i].bajas_temperaturas + estados[i].sistema_salud + nuevo_virus.densidad*estados[i].
densidad + nuevo_virus.ganaderia*estados[i].ganaderia + nuevo_virus.pobreza*estados[i].pobreza + nuevo_virus.respiratorio*estados[i].
respiratorio

        resistencia.append( int(resis))

```

```

#Normalizamos

```

```

denominador = max(resistencia)
min_norm = min(resistencia)

```

```

for i in range(len(estados)):

```

```

    n = (resistencia[i]-min_norm)/denominador
    if n>=.99:
        n = .99
    elif n<=.7:

```

```

n = .7

estados[i].ind_contagio = n

#Organizmos los vecinos de cada estado según su resistencia, para que se contagie primero el más débil
for k in mexico_grafo.keys():
    mexico_grafo[k].sort(key = lambda i: resistencia[i]) #menor a mayor

# -----
# Simulación con mapa
# -----

# -----
# Combinacion lineal
# -----

#Aquí calculamos la resitencia que tiene cada estado al virus con base en los atributos del estado y que
#favorece la propagación de ese virus en específico. Creamos una combinación lineal

st.header("Simulación")

st.subheader("Distribución poblacional en el mapa")

#Te regresa las coordenadas de las fronteras que importamos de github
def cone(estado):
    geom = estado.geometry
    return geom

#Generamos n puntos dentro de la frontera que nos da geom
def generar_puntos_en_poligono(geom, n):
    puntos = []
    minx, miny, maxx, maxy = geom.bounds
    while len(puntos) < n:
        x = random.uniform(minx, maxx) #Un número aleatorio para que se generen los puntitos uniformemente
        y = random.uniform(miny, maxy)
        p = Point(x, y)
        if geom.contains(p):
            puntos.append(p) #Se guarden los puntos y que estén dentro d ela frontera
    return puntos

Puntos_sanos = []
Puntos_enfermos = []

#Cada vez que se llama la función, chequea cuántos puntos sanos hay en cada estado, cuántos enfermos,
# los genera y los guarda en las dos listas de arriba. Imprime el mapa con los puntos y vacía las listas para poder usar después
def Puntos():
    #Tenemos la lista de estados con la estructura que creamos y una lista de estados que nos da lo que importamos de github
    #Para poder relacionarlas usamos el ID, están en el mismo orden solo que el de ellos empieza en 1 y el de nosotros en 0
    for i in range(len(gdf)):
        estado_gdf = gdf.iloc[i] #github
        estado_id = estado_gdf.id #Nuestra lista

        m = max(0, estados[int(estado_id) - 1].enfermos) #cantidad de personas enfermas en este estado
        n = max(0, estados[int(estado_id) - 1].sanos) #cantidad de personas sanas en un estado

        geom = cone(estado_gdf) #frontera
        puntos_e = generar_puntos_en_poligono(geom, m)
        puntos_s = generar_puntos_en_poligono(geom, n)
        Puntos_sanos.extend(puntos_s)
        Puntos_enfermos.extend(puntos_e)

    fig, ax = plt.subplots(figsize=(10, 10))
    gdf.plot(ax=ax, color='lightgray', edgecolor='black', linewidth=0.3) #Muestra la imagen del mapa
    #Plotea puntos sanos y puntos enfermos respectivamente
    ax.scatter([p.x for p in Puntos_sanos], [p.y for p in Puntos_sanos], color='green', s=5, alpha=0.6, label = 'Población sana')
    ax.scatter([t.x for t in Puntos_enfermos], [t.y for t in Puntos_enfermos], color='red', s=5, alpha=0.6, label = 'Población
contagiada')
    ax.set_title(f"Distribución de población enferma con virus {nuevo_virus.nombre}. Semana: {tiempo}. ")
    plt.legend()
    st.pyplot(fig)

#Vaciamos las listas para poder volver a correr la función
Puntos_sanos.clear()
Puntos_enfermos.clear()

##----- BFS

visitados=[] #Guardamos los IDs de los estados infectados
tiempo = 0 #La cantidad de semanas desde que han pasado desde que empecé a propagarse el virus

#nuevo_virus.paso_semanas cada cierta cantidad de iteraciones cuentan como una semana y ya después imprimimos el mapa

```

```

#Con esta búsqueda de amplitud simulamos como se va esparciendo la enfermedad. Es decir, un estado va contagiando a sus vecinos
#Además nos permite recuperar la trayectoria del virus desde el inicio hasta un estado en particular
def bfs(inicio):#global es para poder usar las funciones que definimos antes de usar esta función
    global mexico_grafo
    global tiempo
    global paso_semanas
    global visitados

#Mismo procedimiento visto en clase
#Señalamos las diferencias. Inicializamos el hilo cada que se infecte un estado, para que se contagie la población
    cola = deque()
    cola.append(inicio)
    estados[inicio].camino = estados[inicio].nombre

    it = 0
    while cola:

        nodo_actual = cola.popleft()

        if nodo_actual in visitados:
            continue

        visitados.append(nodo_actual)

#Cada cierta cantidad de iteraciones cuenta como una semana e imprimimos el mapa actualizado
        if it% nuevo_virus.paso_semanas == 0:# el % es para señalar que cada que las iteraciones sean un multiplo entero de paso_semanas
            Puntos()
            tiempo+=1

            time.sleep(1) #para que las semanas coincidad con el tiempo que pasa en los hilos
#Ya agregamos el estado a visitados, dentro de el vamos a guardar la semana en la que se contagio y comenzar el hilo para que se infecte la
población

            estados[nodo_actual].tiempo_contagio = tiempo
            estados[nodo_actual].iniciar_hilo(nuevo_virus)

            for vecino in mexico_grafo[nodo_actual]:
                if vecino not in visitados:
                    cola.append(vecino)
                    estados[vecino].camino = estados[nodo_actual].camino + " -> " + estados[vecino].nombre

            it+=1

#Si el mapa ya se termino de infectar, lo imprimimos aunque todavia no acabe la semana.
# #La condicional es para no imprimir dos veces, aún así no sea multiplo de 5

            if it%nuevo_virus.paso_semanas == 0:
                tiempo+=1
                Puntos()

#La BFS va a terminar una vez que el virus llegue a todos los estados
# #Detenemos los hilos porque la información dejará de ser relevante a partir de este punto
            Estado.detener_hilos()

#Empezamos la BFS con el estado elegido por el usuario
        bfs(nuevo_virus.estado_inicial)
        st.info("Se ha registrado al menos un caso en cada estado.")
        #de detiene cuando la enfermedad ya llegó a todos los estados

#Mostramos una lista desplegable en la que el usuario puede seleccionar un estado y le mostramos la trayectoria que siguió el virus desde que
inicio

        if st.session_state.virus_actual is not None:
            opciones_estados = {f"{estado.id}. {estado.nombre}": estado.id for estado in estados}
            seleccion_estado = st.selectbox("Seleccione el estado para mostrar el impacto del virus: ", opciones_estados.keys())
            estado_mostrar = int(opciones_estados[seleccion_estado])

            st.info(f"El camino de propagación del virus al estado seleccionado es: {estados[estado_mostrar].camino}")#Info muestra la
información de un cuadrito

            #Creamos una regresión lineal a partir de la cantidad de personsas enfermas en cada tiempo
            #Habíamos guardado la información en historial
            #x = np.linspace(0, tiempo+1, 1)
            x = np.arange(len(estados[estado_mostrar].historial))
            #y es estados[seleccion_estado].historial

            #entrenamos el modelo para generar un polinomio de grado 5
            #####
            # CAMBIO DE COPILOT coef = np.polyfit(x, estado[seleccion_estado].historial , 5)
            coef = np.polyfit(x, estados[estado_mostrar].historial , 5)

            #construye el polinomio con los coeficientes datos
            poly_func = np.poly1d(coef)

            #Calculamos los valores de y en el polinomio
            y_fit = poly_func(x)

```

```

fig, ax = plt.subplots()
ax.scatter(x, estados[estado_mostrar].historial, label="Datos simulación", alpha=0.7)
ax.plot(x, y_fit, color="green", label="Fit polinomial")
ax.legend()
ax.grid()
ax.set_title(f"Personas contagiadas con {nuevo_virus.nombre} en {estados[estado_mostrar].nombre}")
ax.set_xlabel(f"Semana - {estados[estado_mostrar].tiempo_contagio} (semana en que se contagió el estado)")
ax.set_ylabel("Personas enfermas (escala 1/ 100000)")
st.pyplot(fig)

```

#Damos la opción de que el usuario vea los atributos del virus que el creo

```

if st.button("Mostrar información del virus"):
    virus = st.session_state.virus_actual
    st.write(f"### Información del virus '{virus.nombre}':") #Imprime la información que deseas en forma de texto, no es un cuadroito
    st.write(f"- Lluvias intensas: {virus.lluvias_intensas}")
    st.write(f"- Altas temperaturas: {virus.altas_temperaturas}")
    st.write(f"- Bajas temperaturas: {virus.bajas_temperaturas}")
    st.write(f"- Densidad: {virus.densidad}")
    st.write(f"- Ganadería: {virus.ganaderia}")
    st.write(f"- Pobreza: {virus.pobreza}")
    st.write(f"- Respiratorio: {virus.respiratorio}")
    st.write(f"- Estado inicial: {virus.estado_inicial}. {estados[virus.estado_inicial].nombre}")
    st.write(f"- Orden de contagio: {'->'.join(estados[i].nombre for i in visitados)}")

```